

Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN
Advanced Java Programming
BCA — Semester 6 — 2024 — Model Answer Sheet
Subject Code: CACS354

Note: This is a model answer sheet for study purposes. Answers are based on standard curriculum topics.

Group B

Attempt any SIX questions. [6×5=30]

2. What is AWT? Differentiate between AWT and Swing with examples.

Answer: AWT (Abstract Window Toolkit): Java's original GUI framework. Heavyweight components (use OS native resources). Limited components, platform-dependent look. **Swing:** Built on AWT, lightweight components (rendered in Java). Richer component set, pluggable look-and-feel, platform-independent.

Differences: 1) AWT components are heavyweight; Swing are lightweight. 2) Swing has more components (JTable, JTree, JTabbedPane). 3) Swing supports MVC architecture. 4) AWT has platform-dependent look; Swing is consistent. 5) Swing requires javax.swing package.

3. Explain the MVC architecture. How is it implemented in Java web applications?

Answer: MVC (Model-View-Controller): Design pattern separating concerns. **Model:** Business logic and data (JavaBeans, POJOs, database entities). **View:** User interface (JSP, HTML, Thymeleaf). **Controller:** Handles requests, processes input, selects views (Servlets, Spring Controller). **Implementation in Java:** Servlet acts as Controller, receives HTTP requests, calls Model methods, forwards to JSP View. Frameworks like Spring MVC formalize this pattern with annotations (@Controller, @RequestMapping, @ModelAttribute).

4. What is JDBC? Explain the steps to connect a Java application to a database using JDBC.

Answer: JDBC (Java Database Connectivity): API for connecting Java applications to databases. **Steps:** 1) Load driver: `Class.forName('com.mysql.cj.jdbc.Driver')`. 2) Establish connection: `DriverManager.getConnection(url, user, password)`. 3) Create statement: `Connection.createStatement()` or `prepareStatement()`. 4) Execute query: `executeQuery()` for SELECT, `executeUpdate()` for INSERT/UPDATE/DELETE. 5) Process results: Iterate through `ResultSet`. 6) Close resources: Close `ResultSet`, `Statement`, `Connection`. **Types:** `Statement`, `PreparedStatement` (precompiled, prevents SQL injection), `CallableStatement` (stored procedures).

5. Define servlet. Explain the servlet lifecycle with a diagram.

Answer: A Servlet is a Java class that handles HTTP requests and generates responses. It runs on a web server (Tomcat, Jetty). **Lifecycle:** 1) **Loading:** Web container loads servlet class. 2) **Instantiation:** Creates servlet object. 3) **init():** Called once for initialization. 4) **service():** Called for each request. Dispatches to `doGet()`, `doPost()`, etc. 5) **destroy():** Called once before removing servlet. Releases resources. **Advantages:** Platform-independent, efficient (persistent), powerful (full Java API), secure.

6. What is JSP? Differentiate between JSP and Servlet. Explain JSP lifecycle.

Answer: JSP (Java Server Pages): Technology for creating dynamic web content using HTML with embedded Java code. **JSP vs Servlet:** JSP is presentation-focused (HTML with Java); Servlet is logic-focused (Java with HTML). JSP is easier for UI developers; Servlet is better for complex logic. JSP is translated to Servlet internally. **JSP Lifecycle:** 1) Translation: JSP converted to Servlet source. 2) Compilation: Servlet compiled to class. 3) Loading/Instantiation: Class loaded. 4) `init()`: `_jspInit()` called. 5) `service()`: `_jspService()` handles requests. 6) `destroy()`: `_jspDestroy()` for cleanup.

7. Explain Java event handling model with a program to handle button click events.

Answer: Java Event Handling: Based on Delegation Event Model. **Components:** **Event Source:** Component generating events (button). **Event Object:** Contains event info (ActionEvent). **Event Listener:** Interface implementing handler methods (ActionListener). **Process:** 1) Create event source (JButton). 2) Create listener implementing ActionListener. 3) Register listener with source using `addActionListener()`. 4) Implement `actionPerformed()` method. **Example:** `JButton btn = new JButton('Click'); btn.addActionListener(e -> JOptionPane.showMessageDialog(null, 'Button clicked!'));`

8. What is JavaBean? Explain the properties and advantages of using JavaBeans.

Answer: A JavaBean is a reusable software component following conventions: public no-arg constructor, private properties with getters/setters, implements Serializable. **Properties:** Simple (single value), Indexed (arrays), Bound (notifies listeners on change), Constrained (listeners can veto changes). **Advantages:** Reusability across applications, portability (platform-independent), compact and easy to create, supports introspection (development tools can discover properties), can be used in visual builders, supports persistence. **Usage:** In JSP with `<jsp:useBean>`, in Spring as managed beans.

Group C

Attempt any TWO questions. [2×10=20]

9. Explain RMI (Remote Method Invocation) architecture. Write a program to implement a simple calculator using RMI.

Answer: RMI Architecture: Enables Java objects on different JVMs to invoke methods on each other. **Layers:** **Stub/Skeleton Layer:** Stub on client side marshals calls; skeleton on server unmarshals. **Remote Reference Layer:** Manages references to remote objects. **Transport Layer:** Handles actual network communication. **Components:** Remote interface (declares remote methods), Remote object (implements interface), Registry (binds remote objects to names), Client (looks up and invokes). **Calculator Example:** 1) Define Remote interface extending `java.rmi.Remote` with methods like `add(int, int)`. 2) Implement interface extending `UnicastRemoteObject`. 3) Create server, bind to Registry using `Naming.rebind()`. 4) Client uses `Naming.lookup()` to get remote reference. 5) Call methods as if local. **Security:** Requires `SecurityManager` and policy file.

10. What is multithreading? Explain thread lifecycle and synchronization in Java. Write a program to demonstrate thread synchronization.

Answer: Multithreading: Executing multiple threads concurrently within a single process. **Thread Lifecycle:** New → Runnable → Running → Waiting/Timed Waiting → Blocked → Terminated. **Creation:** Extend Thread or implement Runnable. **Synchronization:** Prevents race conditions when multiple threads access shared data. **Methods:** **synchronized keyword:** On methods or blocks. **wait()/notify():** For inter-thread communication. **ReentrantLock:** More flexible locking. **Example:** `class Counter { synchronized void increment() { count++; } }`. Multiple threads calling `increment()` are serialized, ensuring count is accurate. **Deadlock:** Can occur if threads wait for each other's locks. **Best Practices:** Minimize lock scope, use concurrent collections, avoid nested locks.

11. Explain JSP implicit objects and standard actions. Write a JSP program to display employee details from a database using JDBC.

Answer: JSP Implicit Objects: Predefined variables available in JSP without declaration. **request:** HttpServletRequest. **response:** HttpServletResponse. **session:** HttpSession. **application:** ServletContext. **out:** JspWriter for output. **config:** ServletConfig. **pageContext:** PageContext. **page:** current servlet instance. **exception:** Throwable (error pages). **Standard Actions:** (instantiate bean), (set property), (get property), (include resource), (forward request). **Employee Program:** Use , , then JDBC code in scriptlet or JSTL to fetch and display records in HTML table.

These model answers are prepared for educational purposes. Refer to textbooks for comprehensive study.